

Первая система с открытым ключом — система Диффи–Хеллмана

Эта криптосистема была открыта в середине 70-х годов американскими учеными:

- **Уитфилдом Диффи (Whitfield Diffie),**
- **Мartiном Хеллманом (Martin Hellman).**

Она привела к настоящей революции в криптографии и ее практических применениях.



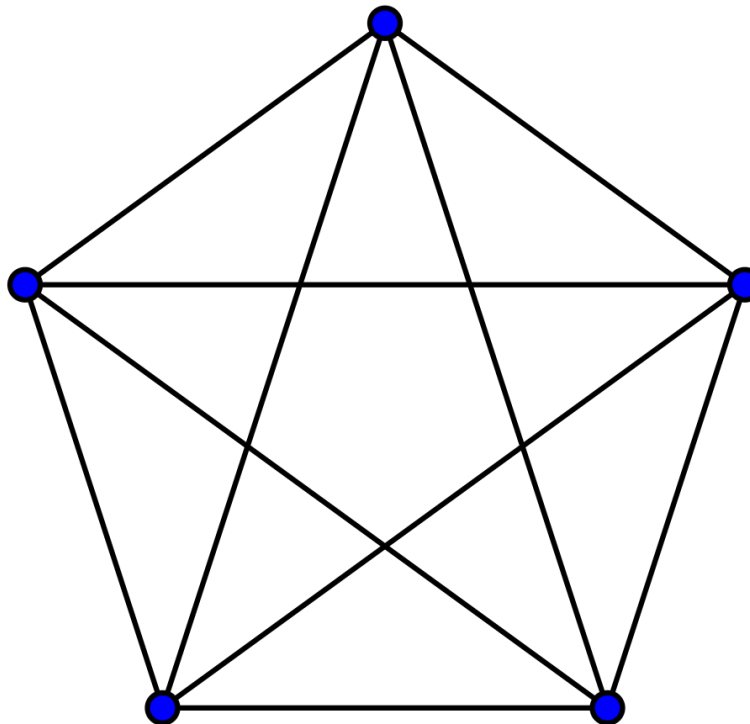
Это первая система, которая позволяла защищать информацию без использования секретных ключей, передаваемых по защищенным каналам.

На рисунке слева направо:

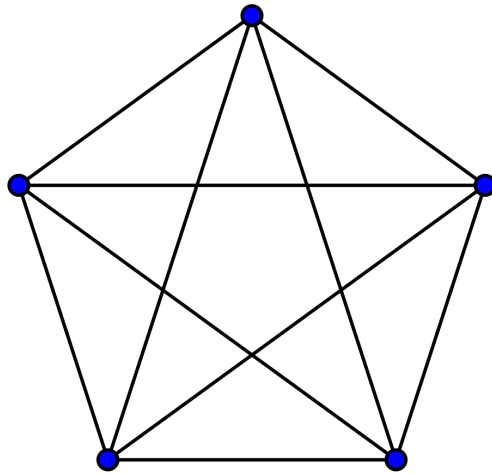
Мартин Хеллман, Уитфилд Диффи

Рассмотрим сеть связи с N пользователями, где N – большое число. Пусть мы хотим организовать секретную связь для каждой пары из них.

Тогда потребуется столько ключей: $C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$



Если мы будем использовать обычную систему распределения секретных ключей, то каждая пара абонентов должна быть снабжена своим секретным ключом, т.е. всего ключей:

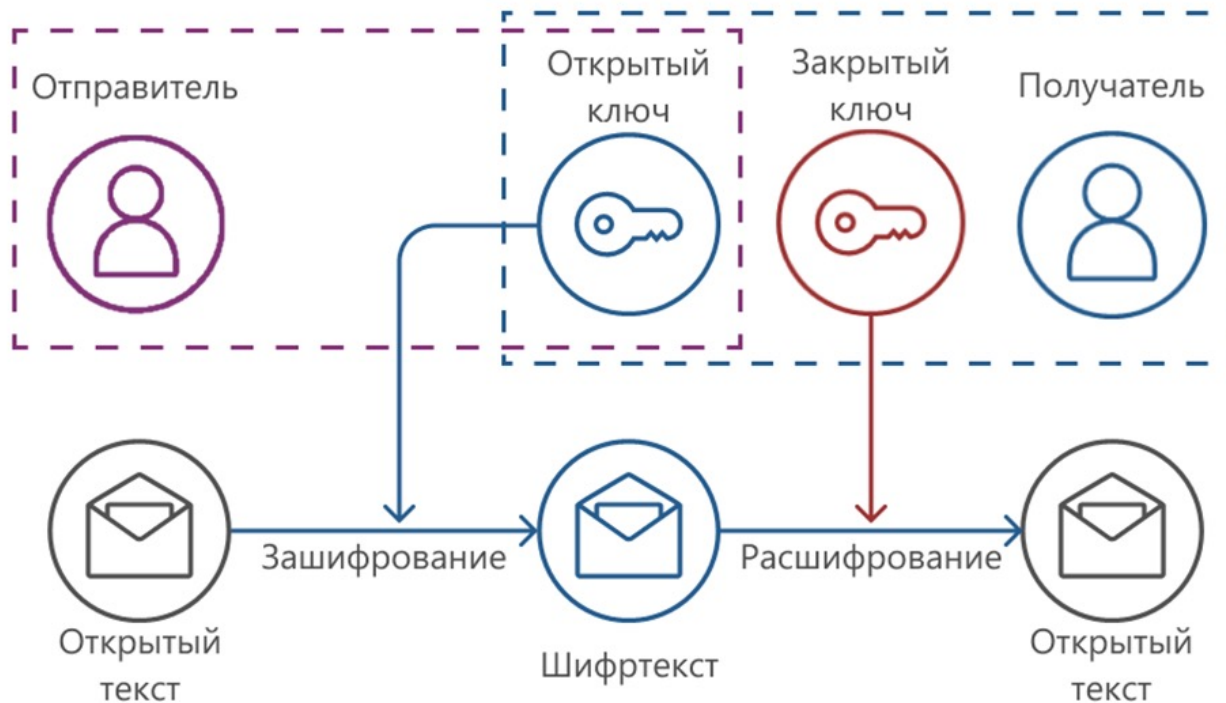


$$C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$$

При большом числе абонентов система снабжения их секретными ключами становится очень громоздкой и дорогостоящей.

У. Диффи и М. Хеллман решили эту проблему за счет открытого распространения и вычисления ключей.

Впервые появилось асимметричное шифрование!



У каждого абонента есть своя открытая и закрытая (секретная) информация.

Схема Диффи-Хеллмана

Строится система для нескольких абонентов: А, В, С.

Общие открытые параметры системы:

p – большое простое число,

g – первообразный корень по модулю p , $1 < g < p - 1$

Секретные параметры абонентов:

X_A, X_B, X_C – большие случайные числа в диапазоне $[2; p - 1]$

Открытые параметры абонентов:

$$Y_A = g^{X_A} \bmod p,$$

$$Y_B = g^{X_B} \bmod p,$$

$$Y_C = g^{X_C} \bmod p.$$

**Т а б л и ц а 2.2. Ключи пользователей в системе
Диффи–Хеллмана**

Абонент	Секретный ключ	Открытый ключ
A	X_A	Y_A
B	X_B	Y_B
C	X_C	Y_C

Допустим, абонент A решил организовать сеанс связи с B , при этом обоим абонентам доступна открытая информация из табл. 2.2. Абонент A сообщает B по открытому каналу, что он хочет передать ему сообщение. Затем абонент A вычисляет величину

$$Z_{AB} = (Y_B)^{X_A} \bmod p \quad (2.10)$$

(никто другой кроме A этого сделать не может, так как число X_A секретно). В свою очередь, абонент B вычисляет число

$$Z_{BA} = (Y_A)^{X_B} \bmod p. \quad (2.11)$$

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Чтобы не проверять все ℓ от 1 до $\varphi(m)$, достаточно проверить три условия:

1. Является ли g числом **взаимно простым** с m , и если нет - то это не первообразный корень.
2. Так как $\varphi(m)$ - **всегда чётное число** для всех $m > 2$, то $\varphi(m)$ имеет как минимум один простой делитель - простое число 2, следовательно, для того, чтобы отсеять значительное количество не-корней, достаточно проверить $g^{\varphi(m)/2} \not\equiv 1 \pmod{m}$ для числа, подходящего на первообразный корень по модулю m .^[1] Если результат $+1 \equiv m$, то g - не корень, в ином случае результат $-1 \equiv m$, когда g - это возможно первообразный корень.
3. Далее следует убедиться, что $g^{\ell} \not\equiv 1 \pmod{m}$ для всех $\ell = \frac{\varphi(m)}{p}$, где p - простой делитель числа $\varphi(m)$, полученный в результате его факторизации.

В практических реализациях для X_A и X_B используются числа порядка 10^{100} и p порядка 10^{300} . Число g не обязано быть большим и обычно имеет значение в пределах первого десятка.

Другими словами, **первообразный корень** — это образующий элемент мультипликативной группы кольца вычетов по модулю m .

Сравнение двух целых чисел по модулю натурального числа m — математическая операция, позволяющая ответить на вопрос о том, дают ли два выбранных целых числа при делении на m один и тот же остаток.

Любое целое число при делении на **m** дает один из **m** возможных остатков: число от 0 до **$m - 1$** .

Это значит, что все целые числа можно разделить на **m** групп, каждая из которых отвечает определённому остатку от деления на **m** .

Эти группы называются **классами вычетов по модулю m** , а содержащиеся в них целые числа — **вычетами по модулю m** .

Арифметические операции с остатками чисел по фиксированному модулю образуют **модульную арифметику**, которая широко применяется в **математике, информатике и криптографии**.

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Число 3 является первообразным корнем по модулю 7. Чтобы в этом убедиться, достаточно каждое число от 1 до 6 представить как некоторую степень тройки по модулю 7:

$$3^1 \equiv 3 \pmod{7}$$

$$3^2 \equiv 2 \pmod{7}$$

$$3^3 \equiv 6 \pmod{7}$$

$$3^4 \equiv 4 \pmod{7}$$

$$3^5 \equiv 5 \pmod{7}$$

$$3^6 \equiv 1 \pmod{7}$$

Задача быстрого нахождения чисел p и g

При произвольно заданном p она может оказаться трудной задачей, связанной с разложением на простые множители числа $p-1$.

Для обеспечения высокой стойкости рассмотренной системы число $p-1$ должно обязательно содержать большой простой множитель (в противном случае можно быстро вычислить дискретный логарифм с помощью алгоритма Полига–Хеллмана).

Поэтому часто рекомендуют использовать следующий подход. Простое число p выбирается таким, чтобы выполнялось равенство

$$p = 2q + 1,$$

где q — также простое число. Тогда в качестве g можно взять любое число, для которого справедливы неравенства

$$1 < g < p - 1 \quad \text{и} \quad g^q \bmod p \neq 1.$$

При работе алгоритма каждая сторона:

1. Генерирует случайное натуральное число — закрытый ключ
2. Совместно с удалённой стороной устанавливает открытые параметры **p** и **g** (обычно значения **p** и **g** генерируются на одной стороне и передаются другой), где **p** является случайным простым числом, а **q = (p-1)/2** также должно быть случайным простым числом (для повышения безопасности).
g является первообразным корнем по модулю p (также является простым числом).
3. Вычисляет открытый ключ, используя преобразование над закрытым ключом

$$Y_A = g^{X_A} \bmod p$$

$$Y_B = g^{X_B} \bmod p$$

4. Обменивается открытыми ключами с удалённой стороной
5. Вычисляет общий секретный ключ **Z**, используя открытый ключ удаленной стороны В и свой закрытый ключ.

На стороне А:

$$\begin{aligned} Z_{AB} &= Y_B^{X_A} \mod p = \\ &= (g^{X_B} \mod p)^{X_A} \mod p = \\ &= (g^{X_B})^{X_A} \mod p = \\ &= g^{X_B X_A} \mod p \end{aligned}$$

На стороне В:

$$\begin{aligned} Z_{BA} &= Y_A^{X_B} \mod p = \\ &= (g^{X_A} \mod p)^{X_B} \mod p = \\ &= (g^{X_A})^{X_B} \mod p = \\ &= g^{X_A X_B} \mod p \end{aligned}$$

Утверждение 2.2. $Z_{AB} = Z_{BA}$.

Доказательство. Действительно,

$$\begin{aligned} Z_{AB} &= (Y_B)^{X_A} \bmod p = (g^{X_B})^{X_A} \bmod p = \\ &= g^{X_A X_B} \bmod p = (Y_A)^{X_B} \bmod p = Z_{BA}. \end{aligned}$$

Отметим главные свойства системы:

- 1) абоненты A и B получили одно и то же число $Z = Z_{AB} = Z_{BA}$, которое не передавалось по открытой линии связи;
- 2) Ева не знает секретных чисел $X_A, X_B, \dots$, поэтому не может вычислить число Z_{AB} (вообще говоря, она могла бы попытаться найти секретное число X_A по Y_A (см. (2.9)), однако при больших p это практически невозможно (требуется миллионы лет)).

Пример 2.2. Пусть $p = 23 = 2 \cdot 11 + 1$ ($q = 11$). Выберем параметр g . Попробуем взять $g = 3$. Проверим: $3^{11} \bmod 23 = 1$ и значит, такое g не подходит. Возьмем $g = 5$. Проверим: $5^{11} \bmod 23 = 22$. Итак, мы выбрали параметры $p = 23$, $g = 5$. Теперь каждый абонент выбирает секретное число и вычисляет соответствующее ему открытое число. Пусть выбраны $X_A = 7$, $X_B = 13$. Вычисляем

$Y_A = 5^7 \bmod 23 = 17$, $Y_B = 5^{13} \bmod 23 = 21$. Пусть A и B решили сформировать общий секретный ключ. Для этого A вычисляет $Z_{AB} = 21^7 \bmod 23 = 10$, а B вычисляет $Z_{BA} = 17^{13} \bmod 23 = 10$. Теперь они имеют общий ключ 10, который не передавался по каналу связи. $\square$

МЕТОДЫ ВЗЛОМА ШИФРОВ, ОСНОВАННЫХ НА ДИСКРЕТНОМ ЛОГАРИФМИРОВАНИИ

Вычисления параметров во многих криптографических шифрах основываются на односторонних функциях.

В качестве примера односторонней функции рассмотрим следующую:

$$y = a^x \bmod p,$$

где p — некоторое простое число (т.е. такое, которое делится без остатка только на себя и на единицу), а x — целое число из множества $\{1, 2, \dots, p - 1\}$. Обратная функция обозначается

$$y = a^x \bmod p \quad (3.1)$$

Обратная функция обозначается

$$x = \log_a y \bmod p$$

и называется *дискретным логарифмом*.

Можно вычислить y , если даны a и x , затратив не более, чем $2 \log x$ операций.

Однако отыскание x по известным a и y , т.е. вычисление дискретного логарифма, — задача намного более трудная.

Достаточно рассматривать только показатели x , удовлетворяющие неравенству $0 \leq x \leq p - 1$.

Обозначим через t_y число операций умножения, необходимых для вычисления y в (3.1) по a и x , и будем для краткости называть t_y временем вычисления. Время возведения в степень по алгоритмам из разд. 2.1 не больше $2 \log x$, причем $x < p$. Отсюда

$$t_y \leq 2 \log p \quad (3.2)$$

при любом показателе степени x .

Задача отыскания x в (3.1) по данным a и y .

Оценим сложность прямого перебора:

Сначала вычислить a^1 и проверить, верно ли равенство $a^1 = y$. Если нет, то проверяем $a^2 = y$, если нет, то $a^3 = y$ и т.д. до a^{p-1} .

В среднем потребуется $(p - 1) / 2$ раз умножать на a и проверять равенство. Таким образом, время прямого перебора:

$$t_{\text{п.п.}} \approx p/2.$$

В методе «шаг младенца, шаг великана» время отыскания **X** существенно меньше:

$$t_{\text{ш.м.ш.в.}} \approx 2 \cdot \sqrt{p},$$

а в методе исчисления порядка это время еще меньше:

$$t_{\text{и.п.}} \approx c_1 \cdot 2^{c_2 \sqrt{\log p \log \log p}},$$

где c_1 , c_2 — некоторые положительные константы.

Чтобы сделать сравнение более наглядным, выразим время вычисления через длину числа p в (3.1). Обозначим эту длину в битах через n . При вычислениях по модулю p имеем $n \approx \log p$. Поэтому порядок трудоемкости (в смысле количества операций) упомянутых алгоритмов будет следующий:

$$\begin{aligned} t_y &\approx n, \\ t_{\text{п.п.}} &\approx 2^{n-1}, \\ t_{\text{ш.м.ш.в.}} &\approx 2^{n/2}, \\ t_{\text{и.п.}} &\approx 2^{c_2 \sqrt{n \log n}}, \end{aligned}$$

где $\approx$ означает «пропорционально».

Метод «шаг младенца, шаг великана»

В открытой литературе этот метод был впервые описан Дэниелом Шенксом (Daniel Shanks) в 1973 году.

Это один из первых методов, который показал, что задача вычисления дискретного логарифма может быть решена значительно быстрее, чем методом перебора.

Описание метода «шаг младенца, шаг великана»:

Шаг 1. Сначала берем два целых числа m и k , такие, что

$$mk > p. \quad (3.3)$$

Шаг 2. Вычислим два ряда чисел

$$y, \quad ay, \quad a^2y, \quad \dots, \quad a^{m-1}y \pmod{p}; \quad (3.4)$$

$$a^m, \quad a^{2m}, \quad \dots, \quad a^{km} \pmod{p} \quad (3.5)$$

(все вычисления проводятся по модулю p).

Шаг 3. Найдем такие i и j , для которых выполняется равенство

$$a^{im} = a^j y. \quad (3.6)$$

Утверждение 3.1. *Число*

$$x = im - j \quad (3.7)$$

является решением уравнения (3.1). Кроме того, целые числа i, j , удовлетворяющие (3.6), существуют.

Доказательство. Справедливость (3.7) следует из приводимой ниже символической цепочки равенств, где все вычисления даны по модулю p , а деление соответствует умножению на обратный элемент:

$$a^x = a^{im-j} = \frac{a^{im}}{a^j} = \frac{a^{im}y}{a^j y} = \frac{a^{im}y}{a^{im}} = y.$$

Докажем теперь, что числа i и j , удовлетворяющие (3.6), существуют. Для этого сведем все числа вида (3.7) в таблицу 3.1.

Т а б л и ц а 3.1. Распределение чисел вида $it - j$

$i \downarrow \quad j \rightarrow$	0	1	2	...	$t - 1$
1	t	$t - 1$	$t - 2$	...	1
2	$2t$	$2t - 1$	$2t - 2$	...	$t + 1$
...	...	...	...	...	...
k	kt	$kt - 1$	$kt - 2$	...	$(k - 1)t + 1$

Мы видим, что в таблице содержатся все числа от 1 до kt . Значит, из (3.3) следует, что в таблице содержатся все числа от 1 до p . Таким образом, любой показатель степени $x < p$ будет содержаться в таблице, т.е. число x , удовлетворяющее (3.1), может быть представлено в виде (3.7) и всегда найдется в таблице, поэтому уравнение (3.6) всегда имеет решение.

Пример 3.1. Найдем решение уравнения $2^x \bmod 23 = 9$, используя метод «шаг младенца, шаг великана».

Выберем m, k . Пусть $m = 6, k = 4$. Мы видим, что (3.3) выполняется. Вычислим числа (3.4), (3.5):

$$(3.4) : \quad 9, 18, 13, 3, 6, 12;$$

$$(3.5) : \quad 18.$$

Дальнейшие вычисления не проводим, так как уже нашлись одинаковые числа в (3.4) и (3.5) при $i = 1, j = 1$. По (3.7) получаем

$$x = 1 \cdot 6 - 1 = 5.$$

Проверим: $2^5 \bmod 23 = 9$. Действительно, $x = 5$ есть решение.

Имея два описанных ранее ряда, можно быстро найти в них равные элементы, воспользовавшись одним из описанных ниже методов.

Метод с использованием сортировки:

Пусть есть два массива $A[m], B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить в этих массивах равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$.

```
int A[m], B[k];
sort(A, A+m);
sort(B, B+k);
int i=0, j=0;
while(i<m && j<k)
{
    if(A[i]<B[j])
        ++i;
    else if(A[i]>B[j])
        ++j;
    else break;
}
if(i<m && j<k)
    cout << i << " " << j;
```

Метод с использованием структур данных:

Пусть есть два массива $A[m]$, $B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$. Либо, в зависимости от используемой структуры данных, с трудоемкостью $O(mC + kC)$, где C - некоторая константа.

```
int A[m], B[k];
map<int,int> dict;
for(int i=0; i<m; ++i)
    dict[A[i]]=i;
for(int i=0; i<k; ++i)
    if(dict.count(B[i]))
    {
        cout << i << " " << dict[B[i]];
        break;
    }
```

Утверждение 3.2. *Время вычислений по данному методу при больших p удовлетворяет неравенству*

$$t_{ш.м.ш.в.} \leq \text{const} \cdot \sqrt{p} \log^2 p. \quad (3.8)$$

(Здесь речь идет о полном времени вычислений, а не о числе умножений).

Доказательство. Мы можем взять

$$k = m = \lfloor \sqrt{p} \rfloor + 1, \quad (3.9)$$

так что, очевидно, (3.3) выполняется. Тогда в (3.4) и (3.5) потребуется не более $2\sqrt{p}$ операций умножения. Мы знаем, что для «обычных» («школьных») методов умножения и деления время вычисления результата для двух r -значных чисел пропорционально r^2 . У нас все числа берутся из множества $\{1, \dots, p\}$, значит, $r \leq \log p$, и время вычисления пропорционально $\log^2 p$. Отсюда мы сразу получаем время, затраченное на вычисление рядов (3.4) и (3.5). Однако мы не

учли все этапы алгоритма. Мы не учли время, требуемое для нахождения равных чисел в этих рядах. При больших k и m это далеко не простая задача. Она может быть решена следующим образом: сначала каждому числу припишем его номер в ряду и еще один бит, в котором указана принадлежность к ряду (3.4) или (3.5), затем преобразуем обе последовательности в список и отсортируем (упорядочим по величине). Длина общего ряда равна $k + m \approx 2\sqrt{p}$. Для лучших методов сортировки требуется $S \log S$ операций сравнения, где S — число элементов в списке (см., например, [1]). В нашем случае $S = 2\sqrt{p}$ и, следовательно, требуется $2\sqrt{p} \log(2\sqrt{p}) \approx \sqrt{p} \log p$ операций сравнения над словами длины $\log p$ бит, т.е. всего требуется порядка $\sqrt{p} \log^2 p$ операций. После сортировки объединенного ряда его надо просмотреть и найти два равных числа из разных рядов (3.4), (3.5), используя битовый признак. Таким образом, суммируя время вычисления на всех этапах, получаем (3.8).